Numerical Analysis

Assignment 1

Mohammed Muddassir

In this report I will attempt to demonstrate how round off error and truncation error affect numerical computations for differentiation and quadrature.

1 Error vs N in Finite Difference Methods

For a test function $u = e^{\sin(x)}$, the exact derivative is $u' = e^{\sin(x)}\cos(x)$. Using finite difference schemes we can approximate this derivative and check how small the absolute error can get as we increase the number of grid-points N. We define the step size h, so we only need to test how N affects the error.

Using the code in "finite-diff-convg.py", I obtained the figure below.

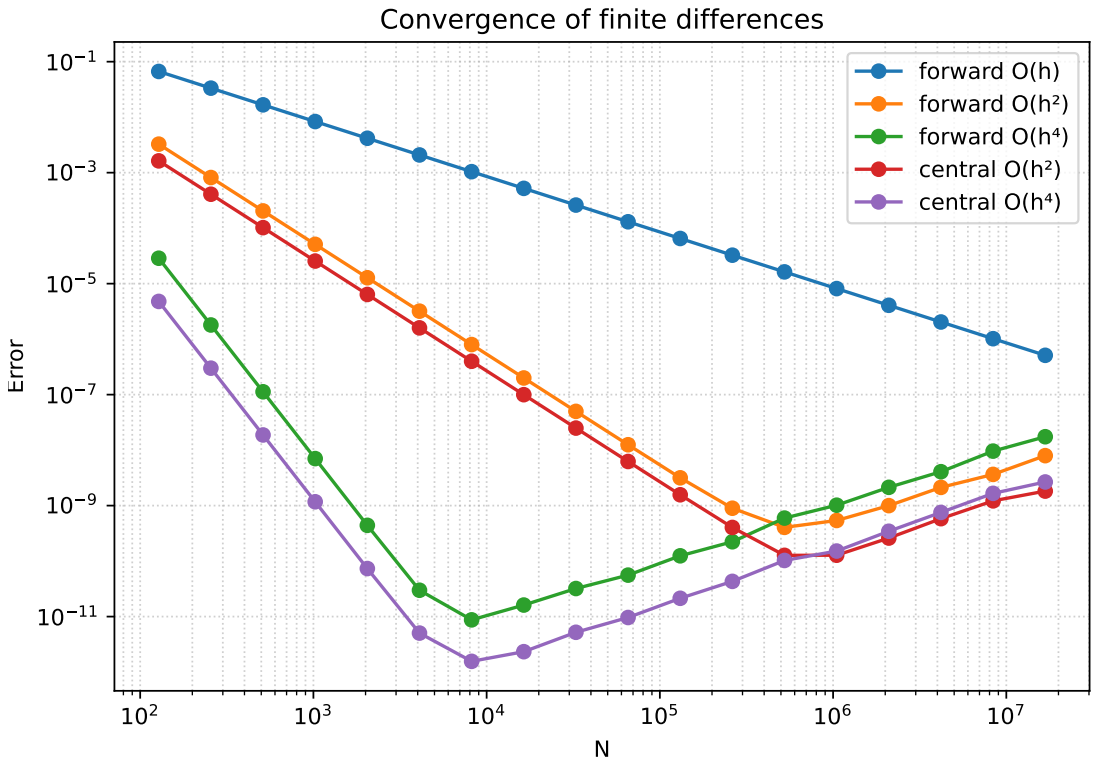


Figure 1: Comparing the error convergence for different finite difference schemes. As expected, the higher order schemes outclass the lower order ones by several orders of magnitude both in terms of the minimum error they're able to achieve, as well as the optimal grid size required to achieve this minimum error. Furthermore, the central approximation schemes achieve a better approximation than the forward schemes of the same order.

Qualitatively, the shape of the graphs is as expected. With relatively large grid spacing, the truncation error dominates, with the error decreasing as N increases. The approximate theoretical slopes are also achieved in these realms. For example, in the second order schemes, the error decreases as a square of the increase in grid-points. A $\times 10$ increase in gridpoints results in approximately a $\times 100$ decrease in error. Similar trends are present for the first and fourth order schemes.

As the grid spacing is decreased beyond the optimal amount however, round off error starts to dominate. We don't see this in the first order scheme because the optimal N is well above 10^7 and it's extremely unlikely for this scheme to be used even if the minimum error was lower than the higher order schemes (which it won't be) because the time it would take to compute would not be worth the difference in error.

Quantitatively, the central finite difference approximation of the fourth order achieves the best results. Reaching a minimum error of $\sim 10^{-12}$ at a relatively small grid size of $N \approx 10^4$. The forward scheme at the fourth order is not far behind but it barely touches $e_N = 10^{-11}$ before the round-off error takes over.

The optimum grids for each of the schemes appear to be as follows

Scheme	**N** for minimum error	Minimum Error
1st order forward (O(h))	beyond $\sim 10^7$	unknown but not less than 10^{-9}
2nd order forward (O(h^2))	$\sim 4.5 \times 10^5$	$\sim 10^{-9}$
2nd order central (O(h^2))	$\sim 10^6$	$\sim 10^{-10}$
4th order forward (O(h^4))	$\sim 7 \times 10^3$	$\sim 10^{-11}$
4th order central (O(h^4))	$\sim 7 \times 10^3$	$\sim 5 \times 10^{-11}$

Something interesting happens around $N = 10^6$. The error due to the second order scheme appears to be lower than the fourth order schemes with the same grid spacing. In the second order scheme, let's say the central, the denominator is $2h$, compared to the fourth order where the denominator is $12h$. Once the round-off error starts to dominate, we lose more and more digits everytime we increase the grid spacing, and that loss is amplified by the small denominator. So eventually, it's not unrealistic to expect the second order scheme to actually perform better than the fourth order since the amplification of the error is not as strong in the second order scheme.

2 Initial Value Problem and Error Convergence for Quadrature

2.1 Exact solution by hand

$$\frac{du}{dt} = \frac{1}{1+u} \text{ for } u(0) = 1$$

$$(1 + u)\, du = dt$$

$$\int (1 + u)\, du = \int 1\, dt$$

$$u + \frac{u^2}{2} = t + C$$

$$u(0) = 1 \text{ (i.e., } t = 0, u = 1\text{):}$$

$$(1) + \frac{(1)^2}{2} = 0 + C$$

$$1 + \frac{1}{2} = C \implies C = \frac{3}{2}$$

$$u + \frac{u^2}{2} = t + \frac{3}{2}$$

$$u^2 + 2u - (2t + 3) = 0$$

$$u = -1 \pm \sqrt{4 + 2t}$$

Only the positive root satisfied the initial conditions so,

$$u(t) = \sqrt{4 + 2t} - 1 \tag{1}$$

2.2 Picard Approximation

The Picard iterative approximation for this IVP is given by

$$u^{(k+1)}(t) = u(0) + \int_{t'=0}^{t} \frac{1}{1+u^{(k)}(t')} dt' \text{ with } u^{(0)}(t) = 1.$$

Using the code in 'picard.py' I obtained the figure below.

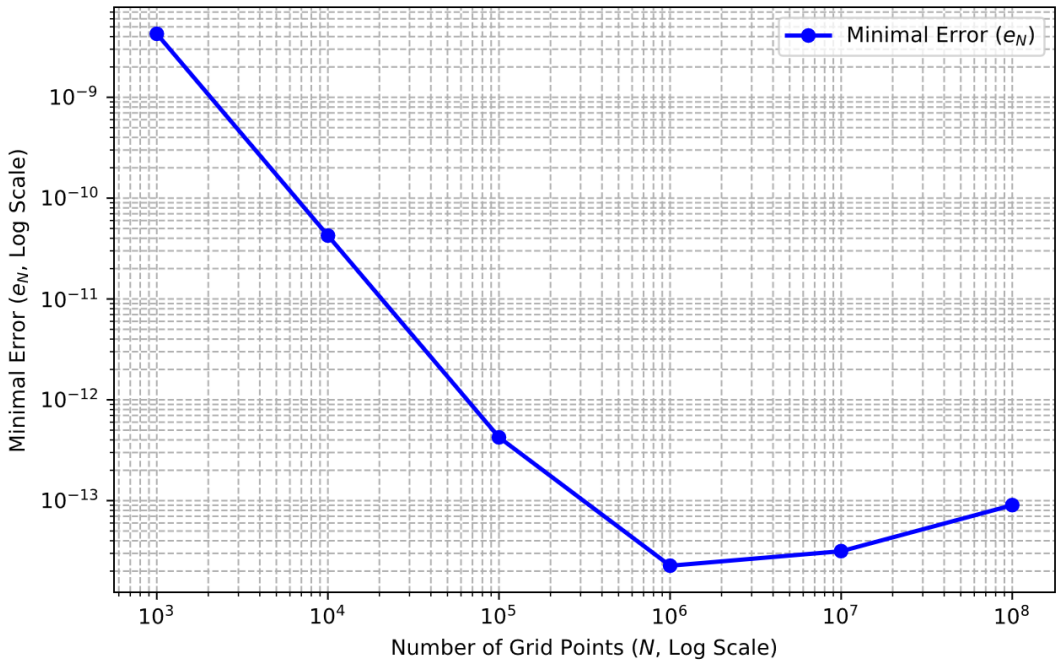


Figure 2: Convergence of error with N for quadrature using the trapezoid rule.

The computation time and array storage capacity exceeded beyond my laptop's capability after $N = 10^8$. However, from the graph, it's clear that I wouldn't need to go farther than that anyway. For the left branch of the plot, the error is $\sim O(h^2)$ which is the expected trend for the trapezoid rule.

I tried to implement Simpson's rule as well, but it took way too long to run, and I'm not enough of an expert to try and implement Gaussian quadrature in code by myself yet. I'm not completely sure it would be a good idea to use Gaussian quadrature anyway considering the time required to calculate the nodes and weights for each N before going to the integral approximation at all.

Similar to the plot for differentiation, there is an uptick in the error after a certain number of grid-points, in this case, $N = 10^6$. The reasoning is similar. For grid sizes with $N > 10^6$, the integral approximation is summing extremely tiny values together, this introduces round-off error that ultimately overcomes the benefit of error reduction that we get from finer grid spacing.

At the lowest point, the minimum error reached $\sim 10^{-14}$ which is very close to machine epsilon. After this point, noise takes over, but as before, the slope on the right branch isn't as steep, which means finer grid spacing is still noticeably effective in reducing error, it's just not strong enough to overcome the round-off error.

I'm sure with a better device I could probably implement Simpson's and the minimum error would be lower than what I got here.